

EcomIQ Technology Stack Reference

Module-by-Module Technology Specifications, Functional Roles, and Selection Rationale

Document Type: Architecture & Technology Reference Manual | **Version:** 2.4.0
Coverage: All 7 System Modules, Libraries, Frameworks, Protocols, and Infrastructure Tools

1. System Technology Stack Visual Map

Below is the complete module-wise technology stack diagram for EcomIQ, mapping every library, framework, database, broker, and protocol to its corresponding container layer.

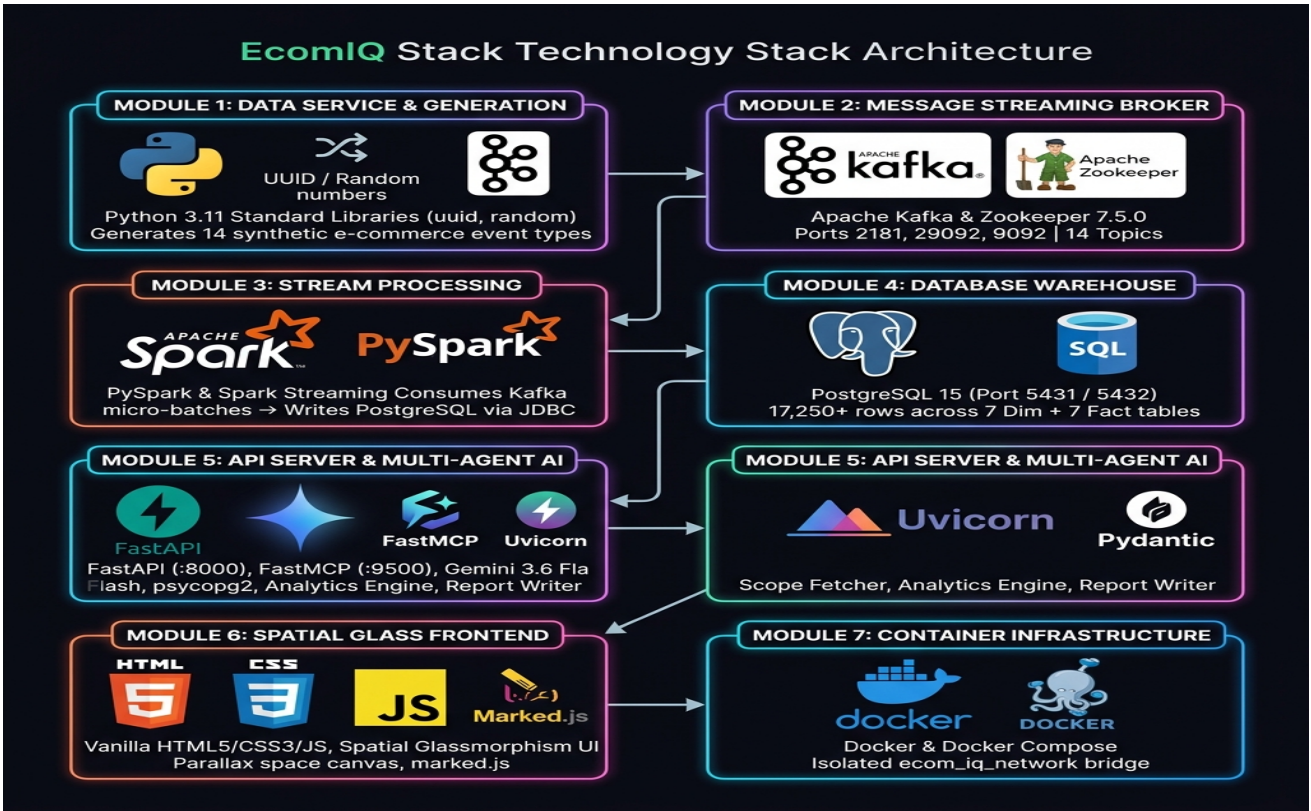


Figure 1.1: EcomIQ Module-Wise Technology Stack Diagram showing all tools, versions, and relationships.

2. Module-by-Module Technology Deep Dive

2.1 Data Generation Module (data_service/)

- **Python 3.11:** Core runtime environment. *Why:* Chosen for rich data science libraries and rapid scripting capability.

- **Python Standard Libraries (uuid, random, datetime):** Built-in generation utilities. *Why: Generates static and transient random transactional values, keys, and timestamps deterministically.*
- **kafka-python:** Apache Kafka client for Python. *Why: Lightweight, pure-Python producer client for publishing JSON records.*

2.2 Message Streaming Broker Module (ecom_iq_kafka & ecom_iq_zookeeper)

- **Apache Kafka 7.5.0:** Distributed event streaming platform. *Why: Provides log immutability, high throughput, and message replay across 14 topics.*
- **Apache Zookeeper 7.5.0:** Cluster coordination service. *Why: Manages Kafka broker state, topic metadata, and leader elections.*

2.3 Stream Processing Module (spark_service/)

- **Apache Spark / PySpark:** Distributed analytics processing framework. *Why: Executes parallel micro-batch transformations over Kafka streams.*
- **Spark Structured Streaming:** Declarative stream processing engine. *Why: Ensures fault-tolerant, exactly-once processing semantics.*
- **PostgreSQL JDBC Driver:** Java Database Connectivity driver. *Why: High-throughput parallel writer connector for PostgreSQL database ingestion.*

2.4 Relational Database Storage Module (ecommerce_iq_db)

- **PostgreSQL 15:** Relational ACID SQL database. *Why: Provides strict relational integrity across 14 tables (7 Dim + 7 Fact) and fast SQL JOINS.*
- **SQL Schema Engine:** Structured Query Language. *Why: Optimized indexes on regions, couriers, payment status, and order IDs across 17,250+ rows.*

2.5 API Server & Multi-Agent AI Module (api_service/)

- **FastAPI 0.140.0:** High-performance Python web framework. *Why: Serves asynchronous REST API endpoints (/investigate) with automatic Swagger docs.*
- **Uvicorn 0.51.0:** Lightning-fast ASGI web server. *Why: Handles asynchronous request routing on port 8000 and port 9500.*
- **Google Gemini API (google-genai):** LLM AI Engine (gemini-3.6-flash). *Why: Extracts JSON search scope from natural language and synthesizes executive reports.*
- **FastMCP 3.4.4:** Model Context Protocol framework. *Why: Exposes 8 specialized analytics tools to external AI agents over SSE (Port 9500).*
- **psycopg2-binary 2.9.12:** PostgreSQL database adapter. *Why: Low-level C-optimized Python driver for raw SQL execution in AnalyticsEngine.*
- **Pydantic:** Data validation library. *Why: Enforces strict JSON schema validation for HTTP requests and AI agent outputs.*

2.6 Spatial Glass Frontend UI Module (frontend/)

- **HTML5 & Vanilla JS (ES6+):** Zero-build web frontend. *Why: Lightweight DOM manipulation without heavy React/Vue framework overhead.*
- **Vanilla CSS3:** Modern styling engine. *Why: Powers iOS spatial glassmorphism, backdrop-filter blur(40px), 3D perspective transforms, and space canvas.*
- **Marked.js:** Client-side Markdown renderer. *Why: Converts AI executive markdown reports into HTML instantly inside floating glass cards.*
- **Google Fonts:** Typography library. *Why: Integrates Google Sans and Google Sans Mono for crisp executive readability.*

2.7 Infrastructure & Deployment Module

- **Docker & Docker Compose:** Container orchestration. *Why: Encapsulates all 6 microservices into isolated environments with unified networking (ecom_iq_network).*

3. Complete Technology Stack Summary Table

Module	Technology	Category	Why Chosen Over Alternatives?
Data Service	Python + Stdlib (uuid, random)	Generator	Fastest developer velocity for generating 14 complex e-commerce JSON schemas using deterministic generators.
Streaming	Apache Kafka	Message Broker	Chosen over RabbitMQ for message replayability and multi-topic partition scaling.
Processing	PySpark	Stream Engine	Chosen over Pandas/Flink for native Spark SQL micro-batching and JDBC sink writers.
Storage	PostgreSQL	RDBMS Database	Chosen over MongoDB for ACID compliance and complex multi-table SQL JOIN analytics.
API Layer	FastAPI + Uvicorn	Web Framework	Chosen over Flask/Django for high-concurrency async performance and Swagger generation.
AI Layer	Google Gemini API	LLM Engine	Chosen for ultra-fast inference speed, large context windows, and low cost.
MCP Server	FastMCP	Agent Protocol	Standardized Anthropic Model Context Protocol allowing external agent connection.
Frontend UI	Vanilla HTML/CSS/JS	Client UI	Chosen over React/Next.js to avoid npm build step and enable instant browser execution.